

WEB APPLICATION SECURITY ASSESSMENT & VAPT REPORT

Web VAPT Training Lab

Target: <http://127.0.0.1:3000>

Prepared by: Bhola Gupta

Assessment Type: Authorized Local Security Assessment

1. Executive Summary

This project demonstrates an end-to-end Web Application Vulnerability Assessment and Penetration Testing workflow against an intentionally controlled local training application. The assessment covered application mapping, reconnaissance, authentication, authorization, session handling, injection testing, XSS, CSRF, API security, security headers, error handling, automated scanning, evidence collection, remediation and retesting.

Seven security findings were documented during the initial assessment. Remediation controls were then implemented and the affected functionality was retested. The current local build passed the documented remediation checks for all seven findings.

2. Scope

Item	Details
Target	http://127.0.0.1:3000
Environment	Kali Linux local laboratory
Application	Node.js / Express training application
Testing Type	Manual + automated supporting validation
Primary Tools	cURL, Burp Suite, Nuclei, browser
Authorization	Controlled local training environment

3. VAPT Methodology

1. Scope Definition
2. Application Mapping
3. Reconnaissance
4. Burp Suite and cURL based manual testing
5. Authentication and authorization testing
6. Injection and XSS testing
7. CSRF and API security testing
8. Security configuration and error handling review
9. Automated supporting scan
10. Evidence collection
11. OWASP mapping and severity assessment
12. Remediation
13. Retesting

4. Findings Summary

ID	Finding	Severity	Status
F-01	Verbose Error / Stack Trace Disclosure	Medium	Fixed

F-02	Missing Authentication on /profile	Medium	Fixed
F-03	Missing Authorization on /admin	Medium	Fixed
F-04	Unauthenticated REST API Exposure	Medium	Fixed
F-05	Invalid Bearer Token Accepted	Medium	Fixed
F-06	Reflected XSS	Medium	Fixed
F-07	CSRF Protection Weakness	Low/Medium	Fixed

5. Detailed Findings

F-01 — Verbose Error / Stack Trace Information Disclosure

Mapping: CWE-209 / OWASP A05

Description: A malformed request originally caused the application to expose a server-side `TypeError` and source-code file path in the HTTP response.

Impact: Information disclosure can reveal implementation details and filesystem paths that assist further testing or exploitation.

Remediation: Generic client-facing error responses were implemented while detailed errors are logged server-side.

Retest: Retest returned HTTP 401 with a generic authentication error and no stack trace.

F-02 — Missing Authentication Enforcement on /profile

Mapping: OWASP A01 — Broken Access Control

Description: The original /profile endpoint returned user information without requiring an authenticated session.

Impact: Unauthenticated users could access protected user information.

Remediation: Session-based authentication and an authentication middleware were implemented.

Retest: Unauthenticated request now returns HTTP 401.

F-03 — Missing Authorization on /admin

Mapping: OWASP A01 — Broken Access Control

Description: The original administrative endpoint was directly accessible without authentication or role validation.

Impact: Administrative functionality could be accessed by unauthorized users.

Remediation: Authentication plus role-based authorization was implemented.

Retest: Unauthenticated users receive 401, normal users receive 403, and admin receives 200.

F-04 — Unauthenticated REST API Data Exposure

Mapping: OWASP A01 — Broken Access Control

Description: The original /api/users endpoint exposed user and role information without proper authorization.

Impact: Sensitive application data could be disclosed to unauthenticated users.

Remediation: Authentication and admin authorization were added to the API endpoint.

Retest: Unauthenticated: 401. Normal user: 403. Admin: 200.

F-05 — Invalid Bearer Token Accepted

Mapping: OWASP A07 — Identification and Authentication Failures

Description: An invalid Bearer token was originally accepted by the API.

Impact: Improper token validation can allow unauthorized API access.

Remediation: Opaque Bearer token validation was implemented using server-side session state.

Retest: Invalid Bearer token now returns HTTP 401. The implementation does not claim JWT signature validation.

F-06 — Reflected Cross-Site Scripting

Mapping: CWE-79 / OWASP A03 — Injection

Description: The comments endpoint originally reflected user-controlled HTML/JavaScript without output encoding. A script payload executed in the browser.

Impact: Successful XSS can execute attacker-controlled JavaScript in the victim's browser.

Remediation: User-controlled output is HTML-escaped before rendering.

Retest: The same payload is now displayed as encoded text and no JavaScript executes.

F-07 — CSRF Protection Weakness

Mapping: CWE-352 / OWASP A01

Description: The original state-changing registration request did not use a CSRF token and accepted a cross-origin-style request.

Impact: An authenticated user's browser could potentially be induced to perform an unintended state-changing action.

Remediation: Session-bound CSRF tokens and SameSite=Strict session cookies were implemented.

Retest: Authenticated request without a valid CSRF token now returns HTTP 403.

6. Security Controls Implemented

- Random session identifiers using cryptographically secure random bytes.
- HttpOnly and SameSite=Strict session cookies.
- Authentication middleware for protected resources.
- Role-based authorization for administrative functionality.
- Opaque Bearer token validation.
- HTML output encoding for reflected user input.
- Session-bound CSRF token validation.
- Content Security Policy.
- X-Content-Type-Options: nosniff.
- X-Frame-Options: DENY.
- Referrer-Policy: no-referrer.
- Generic client-facing error messages.
- Server-side error logging without exposing stack traces.

7. Retesting Results

Finding	Retest	Result
F-01	Malformed request / stack trace	PASS
F-02	Unauthenticated /profile	PASS
F-03	Admin access control matrix	PASS
F-04	REST API authorization	PASS
F-05	Invalid Bearer token	PASS
F-06	Reflected XSS payload	PASS
F-07	Authenticated request without CSRF token	PASS

Retesting result: 7/7 documented findings passed the implemented remediation checks on the current local build. This conclusion is limited to the tested application paths and test cases documented in the assessment.

8. Tests Not Confirmed / Not Applicable

- SQL Injection — tested but not confirmed.
- Classic URL-based IDOR — tested but not confirmed.
- CORS misconfiguration — not confirmed.
- HTTP method misconfiguration — not confirmed.
- Path Traversal — no user-controlled file/path functionality identified.
- Actual File Upload vulnerability — upload functionality disabled/not implemented.
- GraphQL vulnerability — GraphQL attack surface not present in the tested build.

- SSRF — no applicable server-side URL-fetch functionality identified.
- XXE — XXE exploitation not confirmed; verbose XML error handling was remediated.
- Insecure Deserialization — not confirmed.
- HTTP Request Smuggling — no applicable attack surface identified.

9. Evidence

- E-RT-01-F01-verbose-error.txt
- E-RT-02-F02-profile-auth.txt
- E-RT-03-F03-admin-auth-user.txt
- E-RT-03-F03-admin-auth-admin.txt
- E-RT-04-F04-api-auth-unauth.txt
- E-RT-04-F04-api-auth-user.txt
- E-RT-04-F04-api-auth-admin.txt
- E-RT-05-F05-token-validation.txt
- E-RT-06-F06-xss.txt
- E-RT-07-F07-csrf.txt

10. Project Structure

```
web-vapt-lab/
├── README.md
├── vulnerable-web-app/
├── REST-API/
├── screenshots/
├── evidence/
├── notes/
├── report/
└── scans/
```

11. Interview Demonstration Summary

The project demonstrates the complete VAPT lifecycle: Find → Validate → Document → Fix → Retest. During an interview, the tester can demonstrate the local application, show the original security issue, explain the impact, demonstrate the remediation, and show the retest evidence.

Recommended interview flow:

1. Explain application scope and architecture.
2. Demonstrate Burp Proxy and Repeater.
3. Explain authentication versus authorization.
4. Demonstrate XSS testing and output encoding.
5. Demonstrate API authorization.
6. Explain CSRF tokens and SameSite cookies.
7. Show evidence files.
8. Explain remediation and retesting.

9. Close with the complete VAPT lifecycle.

12. Final Conclusion

The assessment successfully demonstrated a practical Web Application VAPT workflow against a controlled local training environment. The project covered manual testing, automated supporting validation, evidence collection, OWASP mapping, remediation and retesting. The final build includes stronger authentication, authorization, session security, CSRF protection, output encoding, security headers and safer error handling.

Security Disclaimer: This project is intended for authorized security testing and training purposes only. Testing should only be performed against systems for which explicit authorization has been obtained.